Exercício 9.3: Depuração: quatro erros "Forbidden" reais

Aluno: Luiz Brito · **Data:** 24/10/2026 · **Disciplina:** Cloud Native Development (CP3)

Ambiente

- Cluster kind com 3 nós, Kubernetes v1.36.1, namespaces `store-dev` e `store-prod`.
- Um usuário por cenário (`user.a` a `user.d`), testado com impersonation (`--as`). Para o RBAC, o resultado é o mesmo de um certificado com esse CN. Sequência completa em `09-commands.sh`.

Armadilha encontrada antes dos cenários: `kubectl auth can-i get pods/log` **não** testa o subresource `log`. O kubectl lê `TIPO/NOME`, então essa linha pergunta "posso dar get no Pod chamado `log`?". Com a Role do cenário A, ela respondeu `yes` mesmo com o `kubectl logs` sendo negado. A forma correta é `--subresource=log`, usada em todo este documento.

```
$ kubectl auth can-i get pods/log -n store-dev --as=user.a
yes
$ kubectl auth can-i get pods --subresource=log -n store-dev --as=user.a
no
```

Cenário A: `kubectl logs` com Role só de `pods`

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: cenario-a
  namespace: store-dev
rules:
  - apiGroups: [""]
    resources: ["pods"]
    verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: cenario-a
  namespace: store-dev
subjects:
  - kind: User
    name: user.a
    apiGroup: rbac.authorization.k8s.io
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: cenario-a
```

Mensagem exata

```
$ kubectl logs -n store-dev pod/sincroniza-catalogo-29880225-kjvd7 --as=user.a
Error from server (Forbidden): pods "sincroniza-catalogo-29880225-kjvd7" is forbidden: User "user.a"
cannot get resource "pods/log" in API group "" in the namespace "store-dev"
```

Sujeito	Verbo	Recurso	apiGroup	Namespace
`User "user.a"`	`get`	`pods/log`	`""` (core)	`store-dev`

Por que falha: `pods/log` é um subresource com autorização própria. Ter `get` em `pods` dá acesso ao objeto Pod (spec e status), não ao log do container. O mesmo vale para `pods/exec`, `pods/portforward` e `pods/attach`.

Correção mínima: só `get` em `pods/log`.

```
   - apiGroups: [""]
     resources: ["pods"]
     verbs: ["get", "list", "watch"]
+  - apiGroups: [""]
+    resources: ["pods/log"]
+    verbs: ["get"]
```

```
$ kubectl auth can-i get pods --subresource=log -n store-dev --as=user.a
yes
$ kubectl logs -n store-dev pod/sincroniza-catalogo-29880225-kjvd7 --as=user.a
sincronizando catalogo
fim

# adjacente continua negado
$ kubectl auth can-i create pods --subresource=exec -n store-dev --as=user.a
no
```

Cenário B: `deployments` no apiGroup core

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: cenario-b
  namespace: store-dev
rules:
  - apiGroups: [""]
    resources: ["deployments"]
    verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: cenario-b
  namespace: store-dev
subjects:
  - kind: User
    name: user.b
    apiGroup: rbac.authorization.k8s.io
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: cenario-b
```

Mensagem exata

```
$ kubectl get deployments -n store-dev --as=user.b
Error from server (Forbidden): deployments.apps is forbidden: User "user.b" cannot list resource
"deployments" in API group "apps" in the namespace "store-dev"
```

Sujeito	Verbo	Recurso	apiGroup	Namespace
`User "user.b"`	`list`	`deployments`	`apps`	`store-dev`

Erro: a Role concede `deployments` em `apiGroups: [""]`, mas esse recurso não existe no grupo core. Deployments ficam em `apps/v1`. O RBAC não valida se o par grupo/recurso existe: a regra é aceita e simplesmente nunca casa. A própria mensagem entrega o grupo certo: `in API group "apps"`.

Correção mínima: trocar só o grupo.

```
-  - apiGroups: [""]
+  - apiGroups: ["apps"]
     resources: ["deployments"]
     verbs: ["get", "list", "watch"]
```

```
$ kubectl auth can-i list deployments.apps -n store-dev --as=user.b
yes
$ kubectl get deployments -n store-dev --as=user.b
No resources found in store-dev namespace.

# adjacente continua negado
$ kubectl auth can-i create deployments.apps -n store-dev --as=user.b
no
```

Cenário C: `kubectl apply` com `create` e `update`

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: cenario-c
  namespace: store-prod
rules:
  - apiGroups: ["apps"]
    resources: ["deployments"]
    verbs: ["create", "update"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: cenario-c
  namespace: store-prod
subjects:
  - kind: User
    name: user.c
    apiGroup: rbac.authorization.k8s.io
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: cenario-c
```

Mensagem exata

```
$ kubectl apply -f 03-deploy.yaml --as=user.c
Error from server (Forbidden): error when retrieving current configuration of:
Resource: "apps/v1, Resource=deployments", GroupVersionKind: "apps/v1, Kind=Deployment"
Name: "store-api", Namespace: "store-prod"
from server for: "03-deploy.yaml": deployments.apps "store-api" is forbidden: User "user.c" cannot
get resource "deployments" in API group "apps" in the namespace "store-prod"
```

Sujeito	Verbo	Recurso	apiGroup	Namespace
`User "user.c"`	`get`	`deployments`	`apps`	`store-prod`

Quais verbos o `apply` exige: com `-v=6` aparecem as requisições HTTP reais.

```
# Deployment ainda nao existe
verb="GET"  url=".../namespaces/store-prod/deployments/store-api" status="404 Not Found"
verb="POST" url=".../namespaces/store-prod/deployments?fieldManager=kubectl-client-side-apply..."
status="201 Created"

# Deployment ja existe (replicas 1 → 2)
verb="GET"   url=".../namespaces/store-prod/deployments/store-api" status="200 OK"
verb="PATCH" url=".../namespaces/store-prod/deployments/store-api?fieldManager=kubectl-client-side-
apply..." status="200 OK"
```

O `apply` sempre começa com `GET` para comparar com o estado atual, e falha nele mesmo quando o objeto ainda não existe. Depois faz `POST` (`create`) se o objeto não existe, ou `PATCH` (`patch`) se existe. Ele **nunca** usa `PUT` (`update`), então o `update` da Role original não serve ao `apply`.

Correção mínima: adicionar só `get` e `patch`. O `update` original foi mantido, porque o enunciado pede para corrigir sem tirar o que já existia.

```
   - apiGroups: ["apps"]
     resources: ["deployments"]
-    verbs: ["create", "update"]
+    verbs: ["get", "create", "update", "patch"]
```

```
$ kubectl apply -f 03-deploy.yaml --as=user.c
deployment.apps/store-api created
$ (replicas 2) kubectl apply -f - --as=user.c
deployment.apps/store-api configured

# adjacente continua negado
$ kubectl auth can-i delete deployments.apps -n store-prod --as=user.c
no
```

Cenário D: RoleBinding de `edit` em `store-dev`, uso em `store-prod`

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: cenario-d-edit
  namespace: store-dev
subjects:
  - kind: User
    name: user.d
    apiGroup: rbac.authorization.k8s.io
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: edit
```

Mensagem exata

```
$ kubectl get pods -n store-prod --as=user.d
Error from server (Forbidden): pods is forbidden: User "user.d" cannot list resource "pods" in API
group "" in the namespace "store-prod"

$ kubectl auth can-i create deployments.apps -n store-dev --as=user.d
yes
$ kubectl auth can-i create deployments.apps -n store-prod --as=user.d
no
```

Sujeito	Verbo	Recurso	apiGroup	Namespace
`User "user.d"`	`list`	`pods`	`""` (core)	`store-prod`

Causa: a ClusterRole `edit` é só um conjunto de regras sem escopo. Quem define o escopo é o binding. Um `RoleBinding` em `store-dev` aplica essas regras só em `store-dev`.

Correção: um segundo `RoleBinding`, idêntico, em `store-prod`, reaproveitando a mesma ClusterRole.

```yaml
# segundo RoleBinding, agora em store-prod, reaproveitando a mesma ClusterRole edit
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: cenario-d-edit
  namespace: store-prod
subjects:
  - kind: User
    name: user.d
    apiGroup: rbac.authorization.k8s.io
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: edit
```

```
$ kubectl get pods -n store-prod --as=user.d
NAME                         READY   STATUS              RESTARTS   AGE
alvo                         1/1     Running             0          96s
store-api-5fc95b9ccf-45lmj   0/1     ContainerCreating   0          0s
store-api-5fc95b9ccf-dz68w   0/1     ContainerCreating   0          0s
$ kubectl auth can-i create deployments.apps -n store-prod --as=user.d
yes

# adjacentes continuam negados
$ kubectl auth can-i create deployments.apps -n default --as=user.d
no
$ kubectl auth can-i create rolebindings -n store-prod --as=user.d
no
```

Por que `ClusterRoleBinding` seria a solução errada: um `ClusterRoleBinding` para `edit` vale em **todos** os namespaces, presentes e futuros: `default`, `kube-system`, `keda`, `store-debug` e qualquer namespace criado depois. O pedido era "operar também em `store-prod`", e o resultado seria escrita em `kube-system`, onde ficam os componentes do cluster, e leitura e escrita de Secrets em todos os namespaces (`edit` inclui Secrets). É o oposto do menor privilégio, e o excesso não aparece em nenhum erro: só se descobre auditando. A forma correta é **um RoleBinding por namespace autorizado**, com a mesma ClusterRole. A lista de namespaces fica explícita nos bindings, e adicionar um terceiro ambiente é uma decisão consciente (um YAML a mais), não um efeito colateral.

Critérios de aceite

Critério	Resultado
Quatro mensagens de erro reais, capturadas do cluster	ok
A: adiciona apenas `pods/log`, `pods/exec` continua `no`	ok
C: adiciona apenas `get` e `patch`	ok, com `-v=6` mostrando GET + POST/PATCH
D: segundo RoleBinding em `store-prod` e justificativa contra `ClusterRoleBinding`	ok